Boosting Tutorial

Idea:
Start with a bunch of training points and fit a weak model.
See where the model goes wrong.
Increase the weight for the 'hard' points, decrease it for the 'easy' ones.
Train the next model on this reweighted data so it focuses more on previous errors.
Repeat this process, and combine all models into a weighted final prediction.

A quick summary:

Assuming binary classification $y \in \{-1, +1\}$, for n sample points $\{x_1, \ldots, x_n\}$.

We'll train weak learners $h : x \to \{-1, +1\}$

Error function: $E(\hat{y}_i, y_i) = e^{-y_i \hat{y}_i}$

For t in $1 \ldots T$:

- Choose $h_t(x)$:
  - Find weak learner $h_t(x)$ that minimizes ϵ_t, the weighted sum error for misclassified points $\epsilon_t = \sum_{\substack{i=1 \\ h_t(x_i) \neq y_i}}^{n} w_{i,t}$
  - Choose $\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \epsilon_t}{\epsilon_t}\right)$
- Add to ensemble:
  - $F_t(x) = F_{t-1}(x) + \alpha_t h_t(x)$
- Update weights:
  - $w_{i,t+1} = w_{i,t} e^{-y_i \alpha_t h_t(x_i)}$ for i in $1 \ldots n$
  - Renormalize $w_{i,t+1}$ such that $\sum_i w_{i,t+1} = 1$
  - (Note: It can be shown that $\frac{\sum_{h_t(x_i)=y_i} w_{i,t+1}}{\sum_{h_t(x_i) \neq y_i} w_{i,t+1}} = \frac{\sum_{h_t(x_i)=y_i} w_{i,t}}{\sum_{h_t(x_i) \neq y_i} w_{i,t}}$ at every step, which can simplify the calculation of the new weights.)

Souce: AdaBoost, https://en.wikipedia.org/wiki/AdaBoost

$H(x) = \text{sign}(F_T(x))$

Why $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$?

Space left below to try working this out.

Consider a binary classification problem with 4 training samples. $y_i \in \{-1, +1\}$
A weak classifier makes the following predictions.

i	y	$h_1(x)$
1	+1	+1
2	+1	-1
3	-1	-1
4	-1	+1

initial weights: $w_i = 0.25$

To do:
- Compute the weighted error ϵ_1
- Compute the classifier weight α_1
- Update the sample weights w_i
- Normalize the updated weights

$\rightarrow$

Space left below to try working this out.

In gradient boosting, gradients are used to identify misclassification.
Unlike in Adaboost, we don't adjust the weights for samples misclassified/correctly classified.
We optimize a loss function of a weak learner iteratively.

1. Start with a base prediction (e.g., the mean of the target values).
2. Calculate the residuals as the differences between actual values and current predictions.
   These residuals represent the error or negative gradient of the loss function.
3. Train a new weak learner to predict those residuals.
4. Update the model by adding this new learner's predictions (scaled by a small learning rate) to the current predictions.
5. Repeat steps 2–4 for many iterations. Each new model focuses on correcting the errors made by the previous ensemble.

```python
def gradient_boosting(X, y, num_iterations=100, learning_rate=0.1, max_depth=3):

    model = []
    current_predictions = [mean(y)] * len(y)

    for iteration in range(num_iterations):

        residuals = [y[i] - current_predictions[i] for i in range(len(y))]

        tree = train_decision_tree(X, residuals, max_depth=max_depth)
        model.append(tree)

        tree_predictions = [tree.predict(x) for x in X]
        for i in range(len(current_predictions)):
            current_predictions[i] += learning_rate * tree_predictions[i]
```

```python
def predict(model, X, learning_rate=0.1):

    predictions = [initial_mean] * len(X)

    for tree in model:
        tree_outputs = [tree.predict(x) for x in X]
        for i in range(len(predictions)):
            predictions[i] += learning_rate * tree_outputs[i]

    return predictions
```

x	y
1	8
2	6
3	10

Q. Predict y from x using gradient boosting
(2 iterations, learning rate = 0.1, depth-1 stumps).

→

Space left below to try working this out.

boosting_demo

April 9, 2026

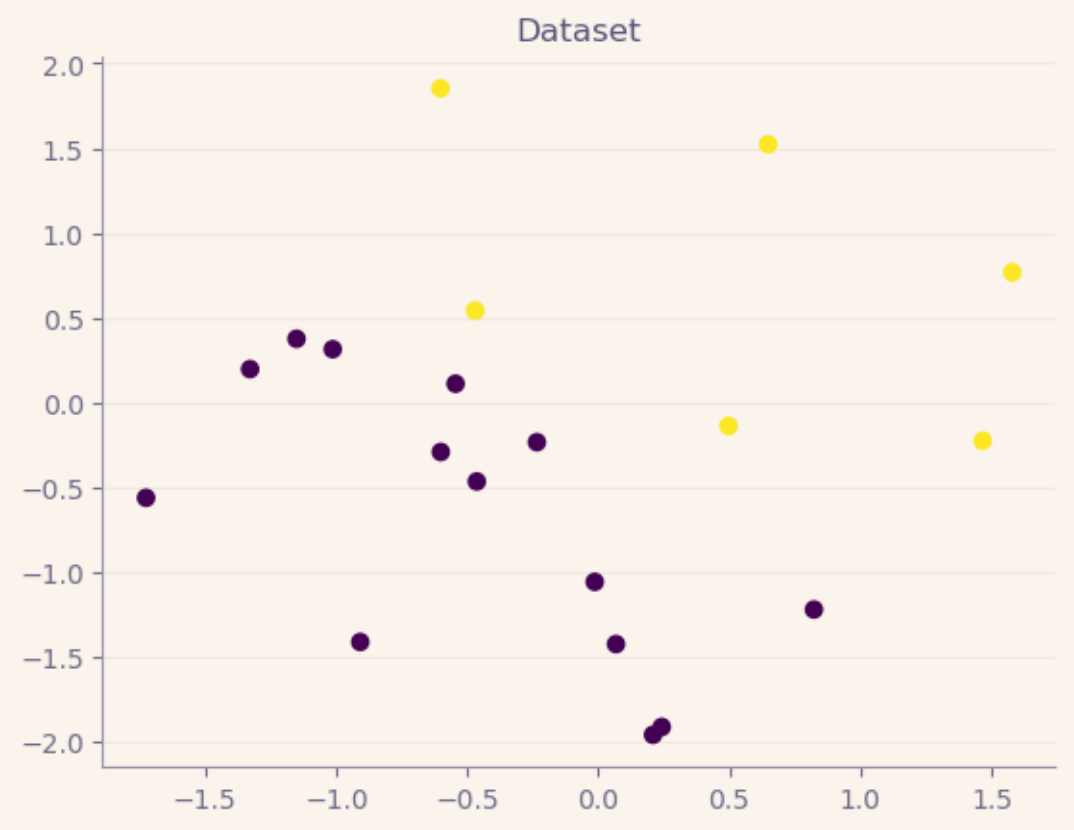
1 AdaBoost Visualization

We visualize how AdaBoost focuses on misclassified points over iterations.

[89]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier

stylesheet = './rose-pine-dawn.mplstyle'
plt.style.use(stylesheet)
```

1.1 Generate Toy Dataset

[90]:
```
np.random.seed(42)
X = np.random.randn(20, 2)
y = np.where(X[:, 0] + X[:, 1] > 0, 1, -1)

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Dataset")
plt.show()
```



1.2 Initialize Weights

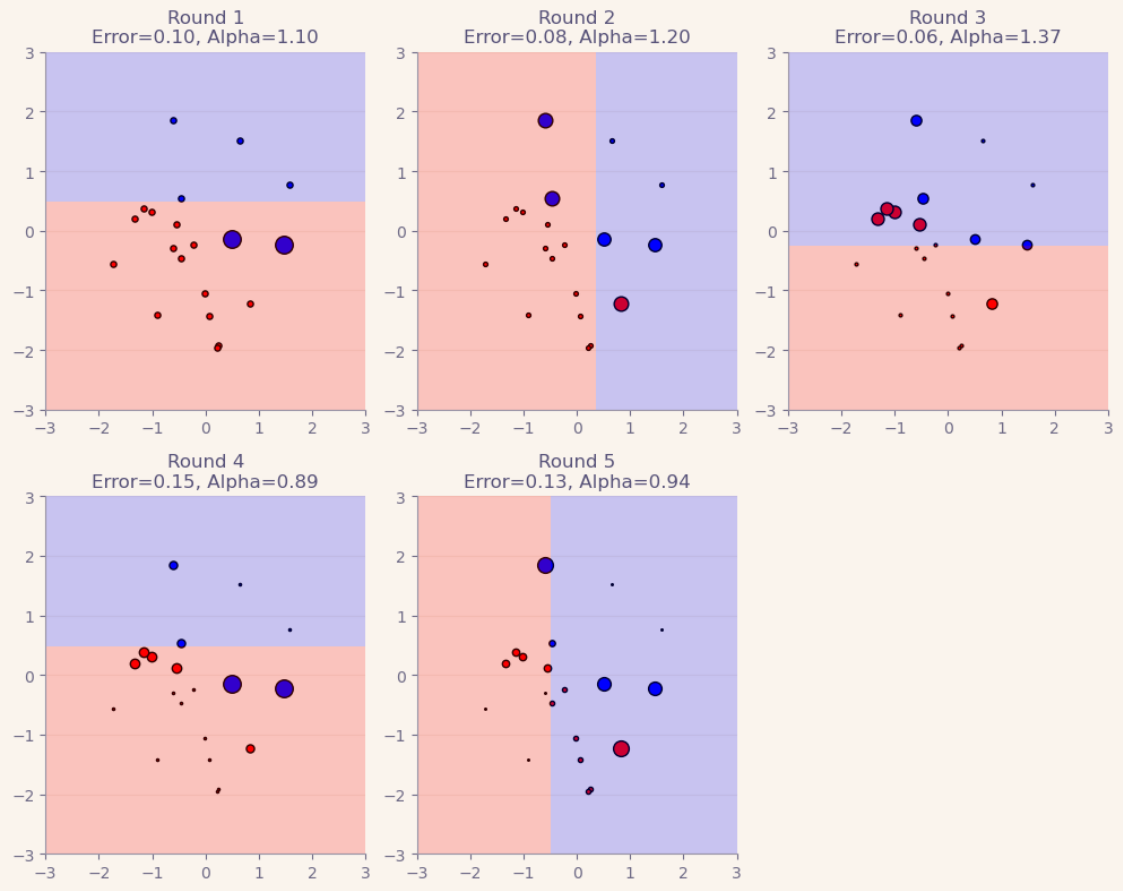
[91]:
```
n = len(y)
w = np.ones(n) / n

print("Initial weights:", w)
```

```
Initial weights: [0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05
0.05 0.05
 0.05 0.05 0.05 0.05 0.05 0.05]
```

1.3 AdaBoost Iterations

[92]:
```
T = 5
models = []
alphas = []

plt.figure(figsize=(10, 8))

for t in range(T):
    stump = DecisionTreeClassifier(max_depth=1)
```

```
    stump.fit(X, y, sample_weight=w)
    pred = stump.predict(X)

    err = np.sum(w * (pred != y))
    err = max(err, 1e-10)

    alpha = 0.5 * np.log((1 - err) / err)

    w = w * np.exp(-alpha * y * pred)
    w = w / np.sum(w)

    models.append(stump)
    alphas.append(alpha)

    # Plot
    plt.subplot(2, 3, t + 1)
    plt.title(f"Round {t+1}\nError={err:.2f}, Alpha={alpha:.2f}")

    sizes = 500 * w

    for i in range(n):
        if y[i] == 1:
            plt.scatter(X[i, 0], X[i, 1], c='blue', s=sizes[i], edgecolors='k')
        else:
            plt.scatter(X[i, 0], X[i, 1], c='red', s=sizes[i], edgecolors='k')

    xx, yy = np.meshgrid(np.linspace(-3, 3, 100),
                         np.linspace(-3, 3, 100))
    Z = stump.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.2, levels=[-1, 0, 1], colors=['red',␣
 ↪'blue'])
    plt.xlim(-3, 3)
    plt.ylim(-3, 3)

plt.tight_layout()
plt.show()
```



1.4 Final Ensemble Prediction

[93]:
```
def final_prediction(X):
    F = np.zeros(len(X))
    for alpha, model in zip(alphas, models):
        F += alpha * model.predict(X)
    return np.sign(F)

y_pred = final_prediction(X)

accuracy = np.mean(y_pred == y)
print("Final Training Accuracy:", accuracy)
```

```
Final Training Accuracy: 1.0
```

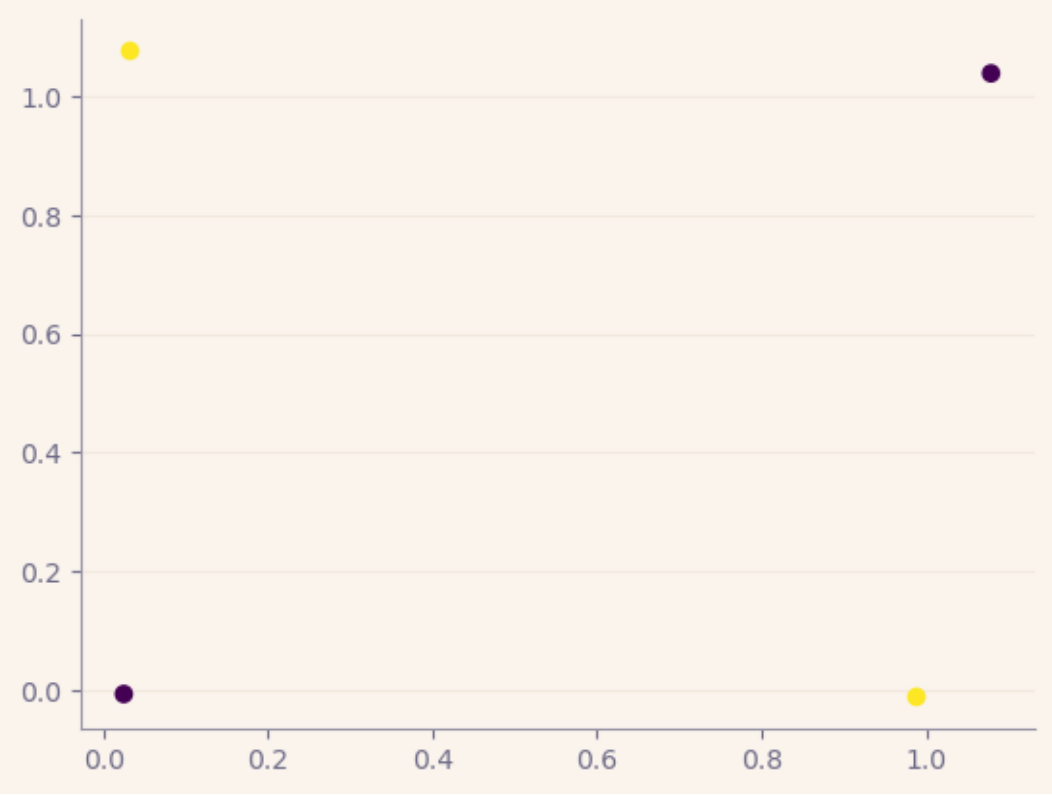
1.4.1 Example:

[94]:
```
from sklearn.tree import DecisionTreeClassifier
```

[95]:
```
np.random.seed(42)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
X = X + 0.05 * np.random.randn(*X.shape)
```

[96]:
```
y = np.array([0, 1, 1, 0])
y = np.where(y == 0, -1, 1)
```

[97]:
```
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.show()
```



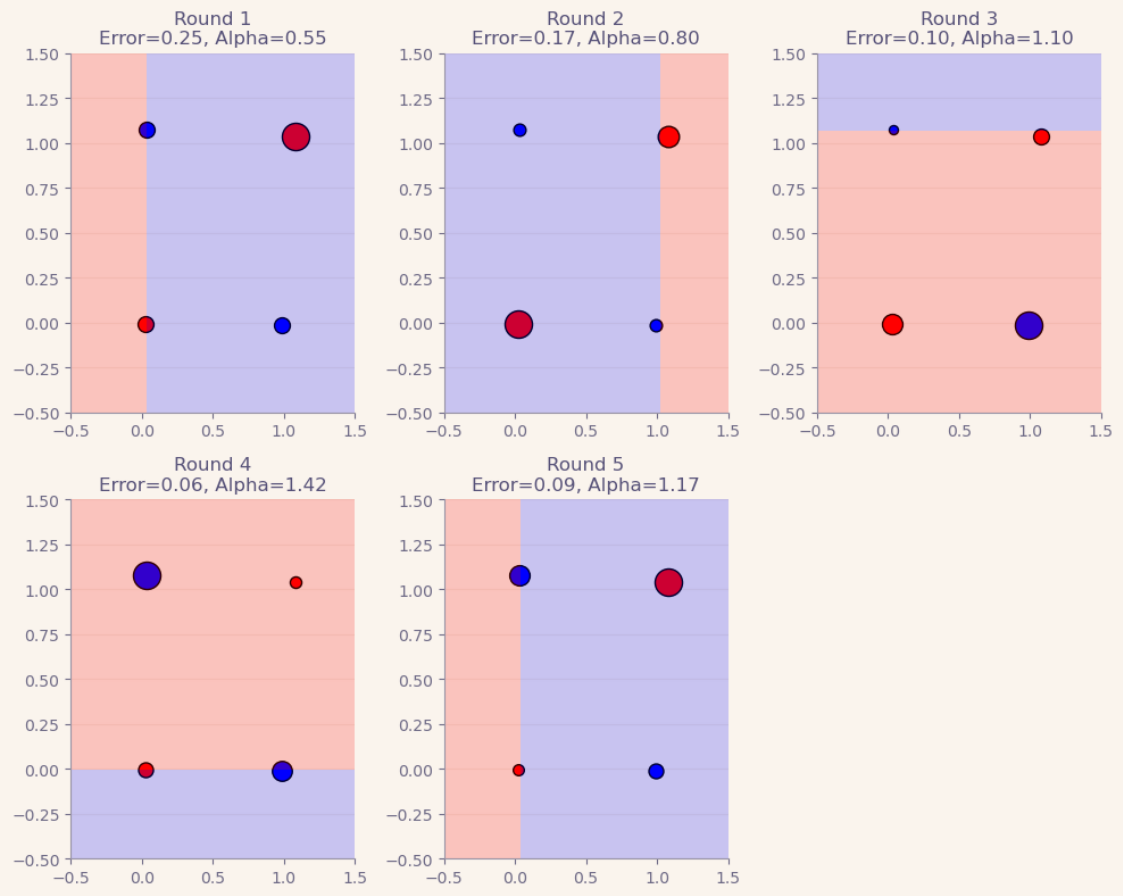
Initialize weights

[98]:
```
n = len(y)
w = np.ones(n) / n
```

[99]:
```
T = 5
models = []
```

```
alphas = []
```

[100]:
```
plt.figure(figsize=(10, 8))

for t in range(T):
    stump = DecisionTreeClassifier(max_depth=1)
    stump.fit(X, y, sample_weight=w)
    pred = stump.predict(X)

    err = np.sum(w * (pred != y))
    err = max(err, 1e-10)

    alpha = 0.5 * np.log((1 - err) / err)

    w = w * np.exp(-alpha * y * pred)
    w = w / np.sum(w)

    models.append(stump)
    alphas.append(alpha)

    plt.subplot(2, 3, t + 1)
    plt.title(f"Round {t+1}\nError={err:.2f}, Alpha={alpha:.2f}")

    sizes = 600 * w

    for i in range(n):
        if y[i] == 1:
            plt.scatter(X[i, 0], X[i, 1], c='blue', s=sizes[i], edgecolors='k')
        else:
            plt.scatter(X[i, 0], X[i, 1], c='red', s=sizes[i], edgecolors='k')

    xx, yy = np.meshgrid(np.linspace(-0.5, 1.5, 100),
                         np.linspace(-0.5, 1.5, 100))
    Z = stump.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.2, levels=[-1, 0, 1], colors=['red',␣
 ↪'blue'])
    plt.xlim(-0.5, 1.5)
    plt.ylim(-0.5, 1.5)

plt.tight_layout()
plt.show()
```



[101]:
```
def final_prediction(X):
    F = np.zeros(len(X))
    for alpha, model in zip(alphas, models):
        F += alpha * model.predict(X)
    return np.sign(F)
```

[102]:
```
y_pred = final_prediction(X)
```

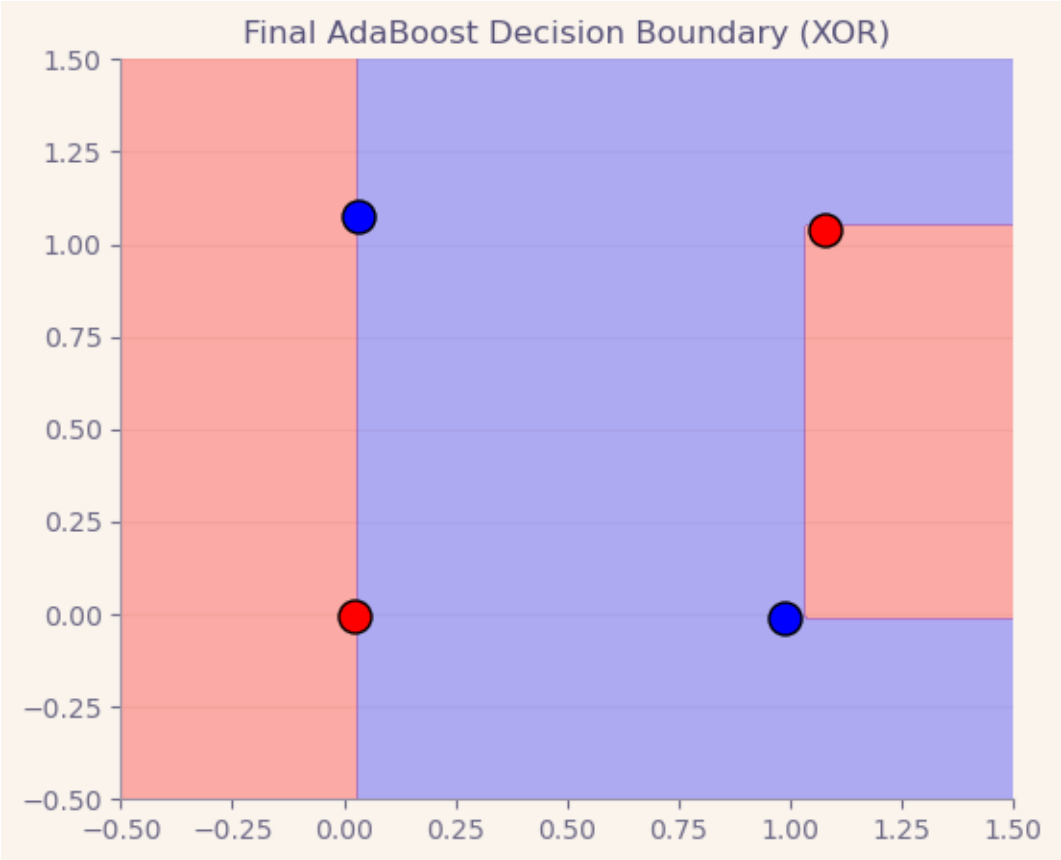
[103]:
```
accuracy = np.mean(y_pred == y)
print("Final Training Accuracy:", accuracy)
```

```
Final Training Accuracy: 1.0
```

Plot final ensemble decision boundary

[104]:
```
xx, yy = np.meshgrid(np.linspace(-0.5, 1.5, 200),
                     np.linspace(-0.5, 1.5, 200))

grid = np.c_[xx.ravel(), yy.ravel()]
```

```
F = np.zeros(len(grid))
for alpha, model in zip(alphas, models):
    F += alpha * model.predict(grid)

Z = np.sign(F)
Z = Z.reshape(xx.shape)

plt.figure(figsize=(6, 5))
plt.contourf(xx, yy, Z, alpha=0.3, levels=[-1, 0, 1], colors=['red', 'blue'])

for i in range(len(X)):
    if y[i] == 1:
        plt.scatter(X[i, 0], X[i, 1], c='blue', s=150, edgecolors='k')
    else:
        plt.scatter(X[i, 0], X[i, 1], c='red', s=150, edgecolors='k')

plt.title("Final AdaBoost Decision Boundary (XOR)")
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.show()
```



grad_boost_numerical

April 9, 2026

Data:

[1]:
```
import numpy as np

X = np.array([[1], [2], [3]])
y = np.array([8, 6, 10])
```

Initialize model:

[2]:
```
F = np.full_like(y, y.mean(), dtype=float)
print("Initial prediction:", F)
```

```
Initial prediction: [8. 8. 8.]
```

0.0.1 Iteration #1:

Calculate residuals

[3]:
```
r1 = y - F
print("Residuals (r1):", r1)
```

```
Residuals (r1): [ 0. -2.  2.]
```

Fit stump

[4]:
```
left_mask = X[:, 0] < 2.5
right_mask = ~left_mask

left_value = r1[left_mask].mean()
right_value = r1[right_mask].mean()

print("Left value:", left_value)
print("Right value:", right_value)
```

```
Left value: -1.0
Right value: 2.0
```

[5]:
```
h1 = np.where(left_mask, left_value, right_value)
print("h1 predictions:", h1)
```

```
h1 predictions: [-1. -1.  2.]
```

Update the model

[6]:
```
eta = 0.1
F = F + eta * h1
print("Updated F after iter 1:", F)
```

```
Updated F after iter 1: [7.9 7.9 8.2]
```

0.0.2 Iteration #2

[7]:
```
r2 = y - F
print("Residuals (r2):", r2)
```

```
Residuals (r2): [ 0.1 -1.9  1.8]
```

[8]:
```
left_value = r2[left_mask].mean()
right_value = r2[right_mask].mean()

h2 = np.where(left_mask, left_value, right_value)
print("h2 predictions:", h2)
```

```
h2 predictions: [-0.9 -0.9  1.8]
```

[9]:
```
F = F + eta * h2
print("Final predictions after 2 iterations:", F)
```

```
Final predictions after 2 iterations: [7.81 7.81 8.38]
```